Marconi Research and Innovations Lab

Infrastructure Operations Agent

Design and Implementation

Abstract

Diagnosing a fault in a small self-hosted environment usually means opening three separate interfaces and correlating them by hand: the container runtime for process state, a metrics server for resource behaviour, and a log store for what the service itself reported. The work is mechanical and has to be repeated for every incident. It also takes longest at the point where an operator can least afford the delay.

This report describes a system that performs that correlation on request. An operator asks a question in ordinary language from a chat client. A language model, given a fixed set of read-only query tools, decides which backends to consult, issues the queries, reads the results, and returns an answer with the evidence it rests on, a likely cause, and recommended next steps for a human.

The design has one governing constraint: the agent can observe but cannot act. No tool in the system issues a mutating call to any backend. The worst outcome of a mistaken model turn is an incorrect answer rather than a stopped service. This constraint is enforced by the set of operations exposed to the model, not by instructions given to it, which is a materially different guarantee.

Beyond the conversational path, the system polls container and endpoint health on a fixed cycle, alerts on transitions, records those transitions durably, and publishes daily and weekly reports. The reports separate measured data, rendered directly from the recorded events and never paraphrased by the model, from generated prose that summarises it. That separation keeps the numbers exact while still giving a reader a short narrative.

The report describes how the system is built and why it is built that way. It closes with the design decisions behind it, what each one costs, and the limitations that remain.

Contents

Abstract **i**

1 Introduction **1**
1.1 The operational problem . . . 1
1.2 Approach . . . 1
1.3 Scope . . . 1
1.4 Out of scope . . . 2

2 System overview **3**
2.1 The conversational path . . . 3
2.2 The monitoring path . . . 3
2.3 The reporting path . . . 3
2.4 Implementation stack . . . 3

3 Architecture **5**
3.1 Layering . . . 5
3.1.1 Foundations . . . 5
3.1.2 Tools . . . 5
3.1.3 Agent and memory . . . 6
3.1.4 Workflows . . . 6
3.1.5 Chat layer . . . 6
3.2 Request flow . . . 6
3.3 Data model . . . 7

4 The agent **8**
4.1 The harness . . . 8
4.2 The middleware stack . . . 8
4.3 The tool budget . . . 9
4.4 Failure handling . . . 9

5 Conversation memory **11**
5.1 Threads . . . 11
5.2 Durability . . . 11
5.3 Bounding a long conversation . . . 11
5.4 Stated limits . . . 12

6 The tool surface **13**
6.1 Composition . . . 13
6.2 Output discipline . . . 13
6.3 Container runtime . . . 13
6.4 Metrics . . . 14
6.5 Logs . . . 14
6.6 Input validation . . . 14
6.7 Read only by construction . . . 14

7 Name resolution **15**

8 Monitoring and alerting **16**
8.1 The poll cycle . . . 16
8.2 Scope . . . 16
8.3 Transition detection . . . 16

8.4 Baselining . . . 16
8.5 Reading state from the runtime rather than querying metrics . . . 17
8.6 Endpoint probing . . . 17
8.7 The event log . . . 17

9 Scheduled reporting **18**
9.1 Schedule . . . 18
9.2 Periodic status . . . 18
9.3 Contents . . . 18
9.4 Downtime accounting . . . 18
9.5 Measured data and generated prose . . . 18
9.6 Publication and follow-up . . . 19

10 The chat interface **20**
10.1 Commands . . . 20
10.2 Conversational access . . . 20
10.3 Output shaping . . . 20
10.4 Log forwarding . . . 20
10.5 Failure must be visible . . . 21

11 Configuration **22**
11.1 Required and optional . . . 22
11.2 Handling of empty values . . . 22
11.3 Loop limits . . . 22

12 Deployment **24**
12.1 Packaging . . . 24
12.2 The observability backends . . . 24
12.3 Networking . . . 24
12.4 Persistence . . . 24
12.5 Runtime socket access . . . 24
12.6 Operating the system . . . 25

13 Verification **26**
13.1 Automated tests . . . 26
13.2 Model behaviour evaluation . . . 26

14 Design decisions **28**
14.1 Read only by construction . . . 28
14.2 The framework's harness, not a hand-written loop . . . 28
14.3 Failure is always answered . . . 28
14.4 Measured data, generated prose . . . 28
14.5 Two storage files . . . 29
14.6 One name-matching rule . . . 29

15 Limitations and extensions **30**
15.1 Limitations . . . 30
15.2 Possible extensions . . . 30

16 Conclusion **31**

References **32**

A Settings reference **34**

B Operation reference **36**

List of Figures

1 Top level arrangement. The agent process is the only component that talks to both the chat workspace and the observability backends. All backend access is read only. 3

2 Layering. Imports run downward only. The agent sits below the workflows that call it and above the tools it is constructed with. 5

3 One investigation. The agent mediates between the model and the backends. The model never reaches a backend directly; it asks the agent to, and reads the result as text. 7

4 The agent as configured. Middleware brackets both nodes: the history is compressed before the model call and the call retried twice on failure; the tool budget is enforced after it, and an operation that raises is retried once before its failure becomes the result text on the return edge. The cycle through the tools and back is the ordinary loop. Not drawn: each model request carries a thirty-second timeout and the whole run a sixty-second one, both outside the graph. 8

5 Transition handling. Failures and recoveries are routed to different channels. The first observation of a target establishes a baseline and produces no alert. 16

1 Introduction

1.1 The operational problem

A small self-hosted environment running on the order of ten services accumulates three separate sources of truth about its own health. The container runtime knows which processes exist, whether they are running, and what exit status they last reported. A metrics server holds numeric time series describing resource consumption and application behaviour. A log store holds the text each service emitted.

Any non-trivial question spans all three. Establishing why a service restarted requires its current state from the runtime, its output immediately before the restart from the log store, and its resource behaviour over the preceding interval from the metrics server. None of these systems answers the question alone, and none of them knows the others exist.

The correlation work is routine. An experienced operator performs roughly the same sequence every time: confirm the exact service name, check whether the process is running, read the recent output, then look at resource curves over the relevant window. The sequence is well defined enough to describe in a paragraph, which makes it a reasonable candidate for automation, but it branches on what each step returns, which is why a fixed script handles it poorly.

1.2 Approach

The system places a language model between the operator and the backends. The operator asks a question in ordinary language. The model has a fixed set of query operations available, each documented with a description written for the model rather than for a human reader. It selects operations, receives their output as text, and either issues further queries or produces an answer.

The model is not asked to know the environment. It is asked to discover it. The available operations include enumeration calls (list the services, list the metric names, list the values of a label) and the guidance given to the model requires discovery before querying. A model that guesses a metric name produces an empty result; a model that enumerates first produces a real one.

Access is through an existing chat workspace rather than a purpose-built interface. This is a deliberate economy. The workspace already has accounts, permissions, notification routing, mobile clients and message history. A question asked in a channel, an alert posted to that channel, and the scheduled report published there all sit in one place, which is where an operator already looks.

1.3 Scope

The system provides four capabilities.

1. **Conversational investigation.** A free-form question is answered by an agent loop with every read-only operation available to it. Context is retained per conversation, so follow-up questions do not need to restate the subject.
2. **Structured incident reporting.** A fixed template drives the same loop toward a report with defined sections: summary, observed behaviour, supporting evidence, likely cause, and recommended next steps.
3. **Continuous monitoring.** Container state and configured HTTP endpoints are polled on a fixed cycle. Transitions produce alerts and durable records.

4. **Scheduled reporting.** Daily and weekly digests summarise current state and the transitions recorded over the reporting window.

1.4 Out of scope

Several things are excluded by design, and knowing which is necessary to read the rest of this report correctly.

1. **No write operations.** The agent cannot restart, scale, reconfigure or delete anything. Where an action is warranted it names the action and leaves it to a human. This is not a temporary limitation pending a permissions model. It is the central design constraint, and Section 6.7 explains how it is enforced.
2. **Single tenant.** The system assumes one environment, one chat workspace, and an operator population that trusts one another. There is no per-user authorisation beyond what the chat workspace itself provides.
3. **No alerting rule engine.** Alerts fire on observed state transitions of containers and endpoints, not on threshold expressions over metric data. Metrics are available to the reasoning loop for diagnosis and are not used to drive alerts. Section 8.5 explains why.
4. **No model training.** No component trains, fine-tunes or otherwise modifies a model. The model is an external service reached over a network API.

2 System overview

The system runs as a single long-lived process holding four concurrent responsibilities: a connection to the chat workspace that receives commands and messages, a reasoning loop that answers questions, a polling loop that watches health, and a scheduled task that publishes reports. Figure 1 shows the arrangement.

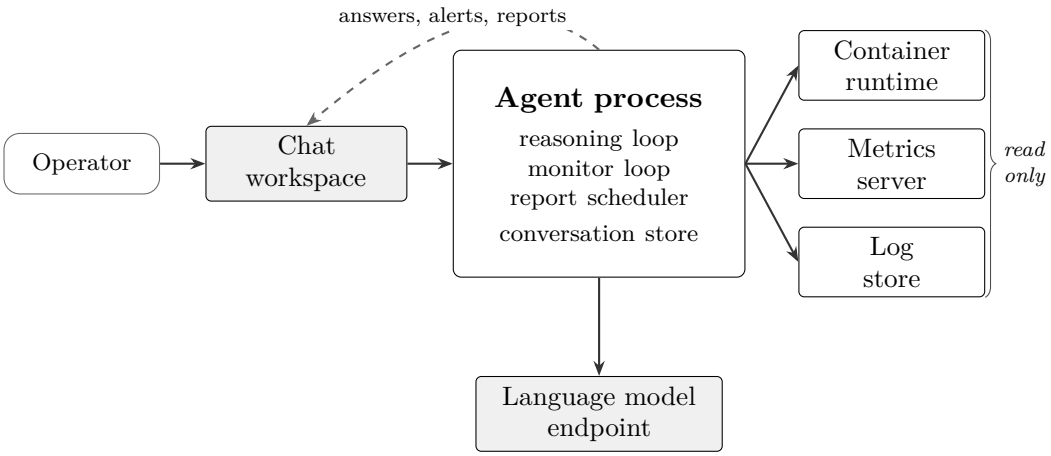


Figure 1: Top level arrangement. The agent process is the only component that talks to both the chat workspace and the observability backends. All backend access is read only.

2.1 The conversational path

An operator issues a slash command, sends a direct message, or mentions the agent in a channel. All three converge on the same reasoning loop. The loop consults the model, executes whatever operations the model requests, feeds the results back, and repeats until the model produces an answer without requesting further operations. The answer is reformatted for the chat platform and sent, split across several messages when it exceeds the platform's per-message limit.

2.2 The monitoring path

Independently of any conversation, a polling task runs on a fixed interval. It reads container state from the runtime and probes each configured HTTP endpoint. It compares what it observes against what it observed on the previous cycle. A change from running to not running, or from reachable to unreachable, is a transition. Transitions are written to a durable event log and posted as alerts. Recovery transitions are posted to a separate channel so that failures and recoveries do not compete for attention in one stream.

2.3 The reporting path

A scheduled task fires once daily at a configured hour. It renders a digest of current state and the transitions recorded in the preceding day, and on one configured weekday it additionally renders a weekly roll-up with per-service outage counts and cumulative downtime. Reports are published to a dedicated channel. Where that channel is a forum, each digest opens its own thread, which keeps a series of reports separable rather than interleaved in one stream.

2.4 Implementation stack

The system is assembled from established components rather than bespoke ones. Table 1 names each and cites its documentation.

Concern	Component	Reference
Runtime	Python 3.13	[1]
Agent harness	LangChain `create_agent` and its middleware	[3]
Conversation store	LangGraph checkpointing	[2]
Model protocol	OpenAI-compatible chat completions	[4]
Chat interface	discord.py over the Discord API	[5, 6]
Configuration	Pydantic settings management	[7]
HTTP transport	HTTPX asynchronous client	[8]
Storage	SQLite	[9]
Container runtime	Docker Engine API	[10]
Orchestration	Docker Compose	[11]
Metrics	Prometheus HTTP API and PromQL	[12, 13]
Logs	Grafana Loki HTTP API and LogQL	[14, 15]
Packaging	uv dependency resolution and locking	[16]
Testing	pytest	[17]
Evaluation	Jupyter notebooks	[18]

Table 1: Implementation stack and the documentation for each component.

3 Architecture

3.1 Layering

The system is arranged in layers with imports running strictly downward, shown in Figure 2. Each layer is defined as much by what it refuses to know as by what it does.

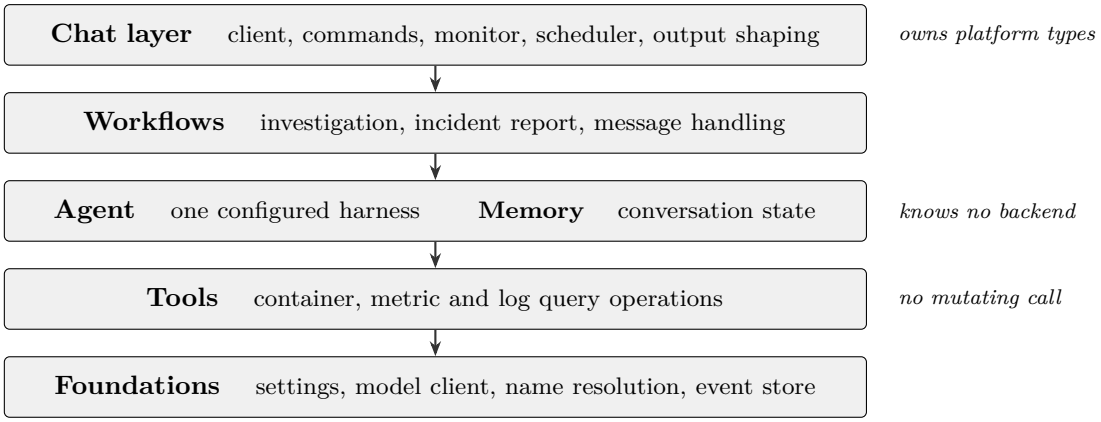


Figure 2: Layering. Imports run downward only. The agent sits below the workflows that call it and above the tools it is constructed with.

3.1.1 Foundations

The settings component reads configuration from the process environment and an optional environment file, exposing it as a validated, typed object constructed once at import. It also resolves both storage paths a single time so that every component agrees on which file is which.

The settings component also derives the model identifier. The configured provider and model name are combined into a single `provider:model` string, which the model library resolves to a client on its own. Because the harness constructs that client internally, configuration reaches it only through the process environment, so the credential and the endpoint address are published there at import. Any server speaking the widely implemented OpenAI-compatible protocol [4] is usable, which is what makes a self-hosted model practical here.

Name resolution provides the single matching rule used for every service lookup in the system. Section 7 describes it.

The event store is a small persistence component over the transition log and the watch list.

3.1.2 Tools

Each tool module pairs a thin asynchronous HTTP client [8] for one backend with the model-callable operations built on top of it. The split matters: the client half is ordinary code that other components call directly, and the tool half adds the schema, the description written for the model, and the clamping of arguments to safe ranges.

This layer knows nothing about conversations, chat platforms or scheduling. It also contains no operation that changes anything.

3.1.3 Agent and memory

One module constructs the agent, and it is the only place in the system where an agent is constructed. It names the model, the tool set, the operations persona, the middleware stack and the conversation store in a single call to the framework's agent factory. The result is built once and cached, and every workflow shares that instance.

Concentrating construction in one place is the most consequential structural decision in the system. A workflow cannot configure the agent differently from any other, so there is exactly one answer to what tools the model has and what instructions it runs under, and that answer is readable in one file. Section 14 states what this costs.

Memory holds conversation state, keyed by thread. It is a peer of the agent rather than a part of it, so that the store's lifetime belongs to the process rather than to any one conversation.

3.1.4 Workflows

A workflow is one unit of work. The investigation workflow binds the full tool set to the operations persona, using the model-facing tool schema abstractions [3]. The incident report workflow calls the investigation workflow with a fixed template. Both are plain asynchronous functions returning a string, with no chat platform types in their signatures, which is what allows the chat layer to remain a thin adapter.

One workflow is an exception worth naming. Message handling for direct messages and mentions necessarily receives platform message objects, because its job is to decide whether an arbitrary incoming message is addressed to the agent at all. It is placed in this layer rather than inside the chat client for a specific reason discussed in Section 10.5.

3.1.5 Chat layer

Everything that knows the chat platform's types lives here: the client and its lifecycle, the command definitions, the monitoring task, the report scheduler, output shaping for the platform's markdown dialect and message length limits, and the bridge that forwards application logs into a channel.

3.2 Request flow

Figure 3 traces one investigation from question to answer.

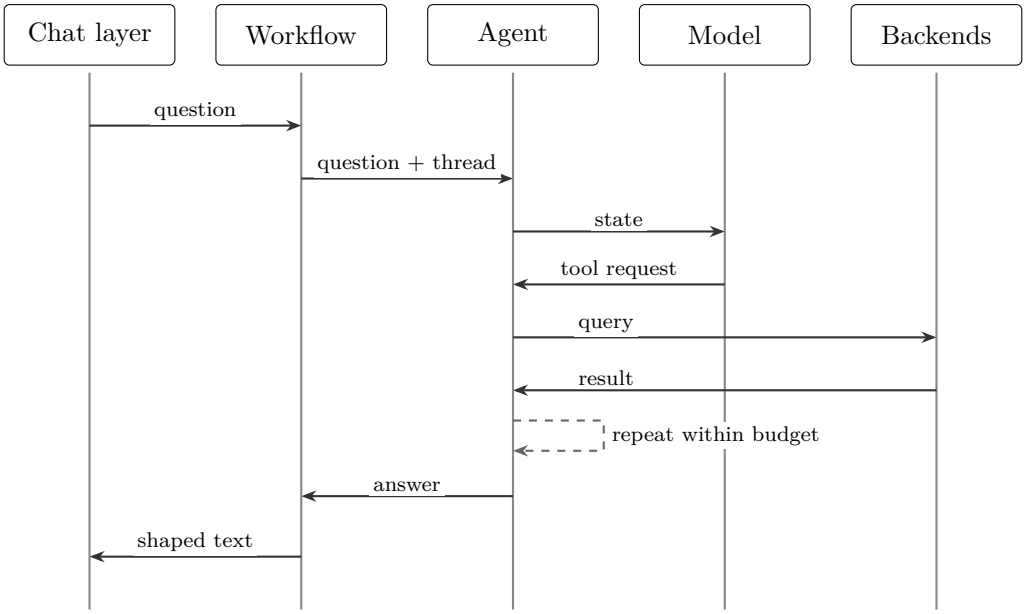


Figure 3: One investigation. The agent mediates between the model and the backends. The model never reaches a backend directly; it asks the agent to, and reads the result as text.

The property worth noting is that the model issues no network calls. It emits a structured request naming an operation and its arguments. The harness looks that name up in the tool set the agent was built with and executes it. An operation that is not in that set cannot be invoked regardless of what the model emits, which makes the tool set an enforcement point rather than a convention.

3.3 Data model

Two embedded database files sit side by side, each opened by exactly one component.

The operational file holds the watch list, naming services an operator has explicitly asked to be monitored regardless of the general scope rule, and the transition log, carrying one row per observed state change with the target, its kind, the new state and a timestamp. Both are read and written synchronously by the event store.

The conversation file holds the checkpoints written by the memory component: the message history for each thread. Nothing else opens it.

The separation matters because the two files are written by different mechanisms: the event store writes synchronously from the polling task, and the conversation store writes asynchronously from the model turn. Sharing one file would put those two in contention for a single write lock, so a watch-list update could stall an answer. Section 14 states what the split costs.

The transition log is trimmed on write, retaining ninety days. A service in a restart loop can otherwise produce one row per polling cycle indefinitely.

4 The agent

4.1 The harness

The agent is one call to the framework's agent factory, made once and cached. The call names five things: the model client, the tool set, the operations persona, a middleware stack, and the conversation store. Everything the loop does between a question arriving and an answer leaving is the framework's, shown in Figure 4.

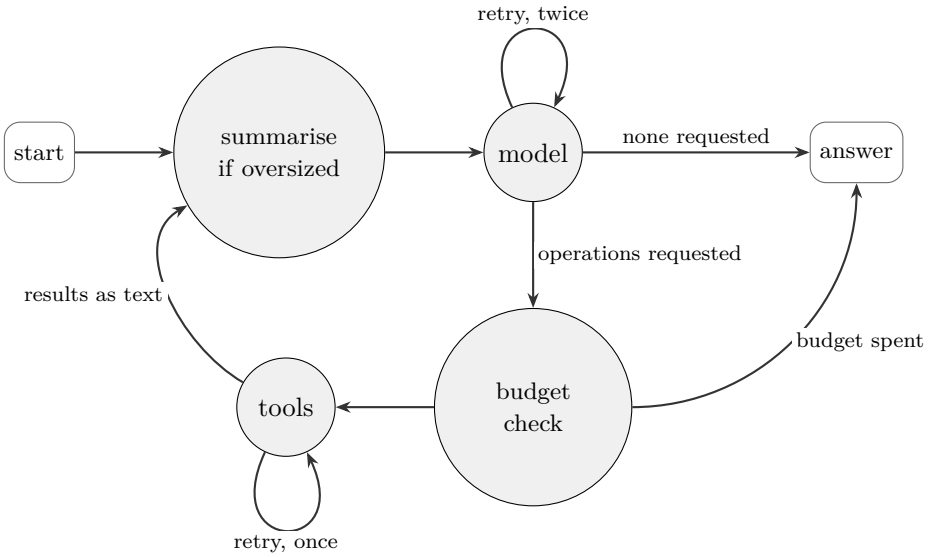


Figure 4: The agent as configured. Middleware brackets both nodes: the history is compressed before the model call and the call retried twice on failure; the tool budget is enforced after it, and an operation that raises is retried once before its failure becomes the result text on the return edge. The cycle through the tools and back is the ordinary loop. Not drawn: each model request carries a thirty-second timeout and the whole run a sixty-second one, both outside the graph.

Termination is decided by whether the model's most recent message requested any operations. If it did, the requested operations run and control returns to the model. If it did not, that message is the answer. There is no separate stopping heuristic and no maximum imposed on the answer itself.

4.2 The middleware stack

Five middleware components are configured. Each replaced code that was previously written by hand in this system, which is the strongest argument for adopting the harness: the behaviours were all needed, and none of them were novel.

1. **Model retries.** A failed model call is retried up to twice before the run is abandoned. Language-model endpoints fail transiently often enough that a single attempt makes the whole system look less reliable than the model it depends on.
2. **Operation retries.** An operation that raises is retried once before its error is handed on. This covers the transient case, a backend that refuses one connection and answers the next, without hiding a backend that is genuinely down: the second failure propagates normally.
3. **Context summarisation.** When the accumulated history exceeds the configured token budget, older turns are replaced with a generated summary before the model is consulted. It may be pointed at a cheaper model than the one answering, since compressing a transcript is

a lesser task than diagnosing an incident.

4. **Tool budget.** A run may spend a fixed number of operations. The call that would exceed the limit is not executed, and neither is any sibling call issued in the same turn; the run ends with an explanation instead.
5. **Operation errors.** An operation that raises is converted into an ordinary result carrying the failure text, which the model reads and continues from.

4.3 The tool budget

Unbounded loops against a paid or rate-limited endpoint are a liability, so the number of operations one question may consume is bounded.

The budget counts operations, not turns

The unit is one operation call. A turn in which the model requests three operations in parallel spends three, not one. Bounding model turns instead would leave the quantity that actually costs money unbounded, since a single turn may request any number of operations. It would also have to be expressed against the shape of the underlying graph, where a change to the graph silently changes the budget.

Exceeding the budget does not produce an error for the operator. The run ends with the model's own account of what it had established, and the partial answer is delivered normally. What the operator loses is the work that was not done, not the work that was.

4.4 Failure handling

Two failure modes are handled inside the loop rather than allowed to propagate. An operation that raises has its exception text returned as the operation's result. An operation the model names but that does not exist is reported as not existing.

In both cases the model receives a description of what went wrong as ordinary input and continues. This is preferable to aborting: a query against an unreachable backend is information the model can use, and it frequently is the answer. A model told that a log query failed because the log store refused the connection can reasonably conclude that the log store is down, which is a correct and useful diagnosis.

Distinguishing the two matters more than it appears. Reporting a non-existent operation as a failed one would tell the model that a capability exists and is broken, when the truth is that it was never offered. That distinction is exactly what the evaluation notebooks in Section 13.2 measure.

Bounded retries sit beneath both. A failing model call is attempted up to three times in total and a raising operation twice, after which the failure is reported as described above rather than retried indefinitely. Retrying without a bound would turn a backend outage into an unbounded stall, which is the failure mode the ceilings below exist to prevent.

Two time ceilings bound the run itself. Each model request carries a thirty-second timeout, and the investigation as a whole is bounded at sixty seconds. A provider that accepts a connection and never answers therefore yields a statement that the investigation timed out, at a predictable moment, rather than holding a chat interaction open until the platform expires it. The ceilings compose: retries may not collectively outlast the outer bound.

Failures above the loop are handled differently. The investigation workflow catches everything

and returns a description of the failure as its answer, so that a caller always receives a string to send.

5 Conversation memory

5.1 Threads

Conversation state is keyed by a thread identifier. The chat layer supplies the identifier of the channel or direct message the question arrived in, so a channel is a conversation. Two channels sharing nothing is what keeps one team's investigation out of another's.

A caller that omits an identifier receives a freshly generated one, giving a one-shot interaction with no history and no persistence beyond the call. Incident report generation uses this path deliberately: a report should describe the service, not whatever was discussed in that channel earlier.

The thread identifier is the only handle on a conversation. There is no interface for reading a stored thread back, writing into one from outside, or deleting one, which keeps the stored history reachable only through the component that owns it.

5.2 Durability

Conversation state is written to its own embedded database file [9], so a thread survives a process restart. An operator who asked a question before a deployment can ask a follow-up after it.

The storage component owns a connection bound to the event loop it was opened in, which requires it to be created inside the running loop rather than at import. It is consequently created on first use and held for the process lifetime. It is also the only opener of that file, which is what makes a single connection sufficient.

Shutdown ordering

The connection runs on a worker thread that continues waiting on its queue after the event loop stops. If it is not closed before shutdown, the process does not exit; it hangs until the supervising runtime gives up and kills it. The chat client's shutdown path closes it explicitly for this reason.

5.3 Bounding a long conversation

A thread grows without bound while a model's context does not. The history is therefore compressed once it exceeds the configured token budget: older turns are replaced by a generated summary, and recent turns are kept verbatim.

Compression rewrites the record, not just the prompt

The summary replaces the older turns in the stored thread, not merely in the message list on its way to the model. The distinction decides whether storage is bounded: a scheme that shortens only the prompt leaves the stored thread holding every message ever exchanged, so a long-lived channel grows a database row without limit. Rewriting in place also discards less than trimming would, since a trimmed thread loses its oldest turns outright while a summarised one keeps a compressed account of them.

Trimming had a second constraint that summarisation removes. A model turn requesting operations and the results of those operations form a unit in the protocol, and delivering one without the other is rejected as malformed, so trim boundaries had to be chosen to keep those pairs intact.

5.4 Stated limits

Memory here means one conversation replayed per thread. There is no recall across threads and no retrieval over past conversations. A fact established in one channel is unavailable in another.

Compression is lossy by construction. A long conversation retains a summary of its early turns rather than the turns themselves, so a detail mentioned once and not repeated may survive only as whatever the summarising model judged worth keeping.

6 The tool surface

6.1 Composition

Ten operations are available to the model, covering three backends. Table 2 summarises them; Appendix B gives the full reference.

Operation	Backend	Purpose
`list_services`	runtime	Enumerate container names and configured endpoints
`docker_status`	runtime	State, status and image of one container
`docker_logs`	runtime	Recent output of one container
`list_metric_names`	metrics	Enumerate available metric names
`get_label_values`	metrics	Enumerate values of a metric label
`query_prometheus`	metrics	Instant query for current values
`query_prometheus_range`	metrics	Range query for behaviour over a window
`get_loki_label_values`	logs	Enumerate values of a stream label
`query_loki`	logs	Log query over a short recent window
`query_loki_range`	logs	Log query over an explicit window

Table 2: The ten read-only operations available to the model.

Four of the ten exist purely for discovery. This proportion is intentional. A model asked about resource usage will otherwise write a query against a plausible-sounding metric name that does not exist in this environment, receive an empty result, and report that usage is zero. Making enumeration cheap and instructing the model to enumerate first converts a confident wrong answer into a correct one.

6.2 Output discipline

Every operation returns text, and every operation bounds what it returns. Query results are truncated to a fixed character count; enumerations are capped at a fixed number of entries. Without these caps a single query against a busy log stream would consume the entire context window and leave no room for the reasoning that follows.

Arguments that describe a range are clamped rather than rejected. A request for twenty-four hours of log data against a six-hour maximum returns six hours, not an error. The model receives usable data and the backend is protected from an expensive query.

6.3 Container runtime

The runtime is queried over its HTTP API [10], reached either through a local socket or a network address. Enumeration returns an empty list when the runtime is unreachable rather than raising, so a question about metrics remains answerable on a host where the runtime is unavailable.

Container output requires one piece of protocol handling. A container without an attached terminal has its output framed: each chunk carries an eight-byte header identifying the stream and giving the payload length. Returning the raw bytes would interleave binary headers with the log text. The framing is parsed and stripped, and remaining non-printable bytes are removed, before the text reaches the model.

6.4 Metrics

The metrics server exposes both instant and range query endpoints over HTTP [12], addressed in its own query language [13]. Instant queries return current values; range queries return a series over a window. The resolution of a range query is derived from the requested window so that the result contains a roughly constant number of points regardless of window length. A thirty-minute window and a six-hour window therefore cost about the same to transmit and to read.

6.5 Logs

Log queries are issued over the log store's HTTP API [14] in its own selector and filter language [15]. They carry one non-obvious behaviour. The instant query endpoint rejects plain stream selectors, accepting only queries that aggregate. Since a selector returning recent lines is exactly what an investigation wants, the instant operation is implemented as a range query over a short recent window. The model sees an instant query and gets the lines it expected.

Label schemas are not shared

The labels attached to log streams are set by the log shipping agent's configuration. They are not the container runtime's names and not the metric server's labels. A model that assumes the three systems share a vocabulary writes selectors that match nothing. The log operations' descriptions state this explicitly and instruct the model to enumerate the real labels before writing a selector.

6.6 Input validation

Label names supplied by the model are placed into a URL path segment when enumerating their values. They are validated against the backends' own permitted character set, letters, digits and underscore, beginning with a letter or underscore, before use. A name failing validation yields an empty result rather than a request.

This is a trust boundary. The model's output is not trusted input: it is generated text influenced by tool results that themselves derive from log lines an arbitrary service produced. Path separators, query and fragment markers, relative path segments and whitespace are all rejected, which prevents a constructed label name from redirecting the request to a different endpoint.

6.7 Read only by construction

The guarantee that the agent cannot act rests on the composition of Table 2 and nothing else. Every operation is a query. None accepts a parameter that would change state, and none is routed to a mutating endpoint.

The instructions given to the model also state that it is read only and should recommend actions to a human rather than claim to perform them. That text improves the quality of answers. It is not the guarantee. Instructions constrain what a model tries to do; the tool table constrains what it can do. A model that ignores its instructions entirely and requests a restart receives a message stating that no such operation exists.

This distinction matters when reasoning about deployment. Section 12.5 returns to it in the context of runtime socket access, where an appealing but misleading alternative guarantee is available.

7 Name resolution

Operators do not type exact container names. A service known conversationally as the API might be deployed under a name carrying a project prefix and a role suffix, and the separator convention is inconsistent across tools: hyphens in one place, underscores in another.

One rule handles this everywhere. A name is reduced by removing separator characters and lowercasing, and a candidate matches if the reduced query appears within the reduced candidate. Under this rule three spellings of the same name are equivalent, and a partial name matches the full one.

Two refinements make it safe.

1. **Exact match wins.** If any candidate's reduced form equals the query's reduced form exactly, that candidate is selected regardless of other partial matches. Without this, a service whose name is a prefix of a longer service's name would become unaddressable.
2. **Ambiguity is an error.** If a partial name matches several candidates and none matches exactly, the lookup fails with a message listing the candidates and asking for a specific one. It does not select the first.

The second refinement is the one that matters operationally. Silently choosing among several matches produces a status report about the wrong service, labelled with the name the operator asked about. That is worse than an error, because it is believable. An explicit ambiguity message costs the operator one round trip and cannot mislead.

The same rule is applied to watch list lookups, so removing a service from monitoring behaves exactly as querying it does.

8 Monitoring and alerting

8.1 The poll cycle

A single task runs on a configured interval with a floor applied, so that a misconfigured interval cannot produce a tight polling loop. Each cycle reads all container states from the runtime and probes each configured HTTP endpoint in turn.

8.2 Scope

Container monitoring is disabled by default and enabled explicitly. When enabled, a container is in scope if its name contains a configured filter substring, or if an operator added it to the watch list through a command. An empty filter places every container in scope.

The union of a broad rule and explicit additions covers the two real cases. An environment where every container matters sets an empty filter. An environment sharing a host with unrelated workloads sets a filter matching its own naming convention and adds anything else that matters individually.

8.3 Transition detection

The task holds the previous observation of each target in memory and compares. A container observed as running and now observed otherwise is a failure transition. The reverse is a recovery. Endpoints follow the same pattern over a reachable flag, where reachable means a response with a non-error status and unreachable means an error status, a connection failure, or a timeout.

Figure 5 shows the resulting behaviour, including the case that produces no output.

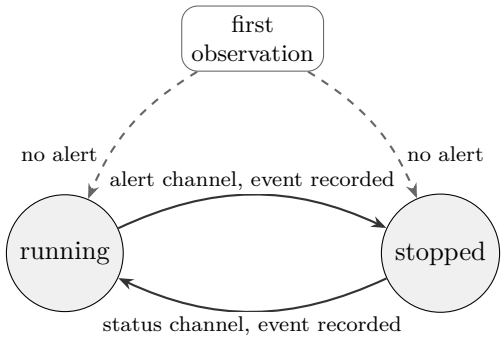


Figure 5: Transition handling. Failures and recoveries are routed to different channels. The first observation of a target establishes a baseline and produces no alert.

8.4 Baselining

The first observation of any target records its state and emits nothing. Only a change from a known previous state is a transition.

Without this, every process restart would announce every container that happens to be stopped and every endpoint that happens to be down. That is precisely the behaviour that teaches operators to mute an alert channel, at which point the entire subsystem has negative value.

The cost is stated honestly: a container that fails while the agent is not running produces no alert and no event. When the agent returns it observes the stopped container, treats that as

a baseline, and stays silent. The failure is still visible in the scheduled report's current-state section, marked as not running, but it is not announced.

8.5 Reading state from the runtime rather than querying metrics

Container liveness could be derived from a metrics query, since the metrics server tracks whether its scrape targets respond. The system reads the runtime directly instead, for two reasons.

1. **Latency.** A metrics-derived signal is delayed by the scrape interval, and the transition it reports is the transition of a scrape, not of a process.
2. **Failure correlation.** If the metrics server itself restarts, every target it scrapes appears to fail simultaneously. An alerting path built on metric queries announces a total environment outage when what actually happened is that the monitoring system bounced. Reading the runtime directly means the metrics server is just another monitored service, and its restart is reported as one service restarting.

Metrics remain fully available to the reasoning loop for diagnosis. The decision here is only about what drives an alert.

8.6 Endpoint probing

Endpoint checks cover what runtime inspection cannot see: a service on another host, a service behind a proxy, or an externally observable health path whose success depends on more than the process being alive. A container can be running while the application inside it returns errors, and only a probe distinguishes those.

8.7 The event log

Every transition is recorded durably with its target, kind, new state and timestamp. The recording happens before the alert is dispatched, so an environment configured with no alert channel still accumulates a complete history. Silencing notifications does not silence the record the reports are built from.

Live state is held only in memory. This is a deliberate asymmetry: the current observation is cheap to rebuild on the next cycle, whereas history cannot be reconstructed. Targets that leave scope have their in-memory state discarded, so a service that returns later is baselined afresh rather than compared against a stale observation from before it left.

9 Scheduled reporting

9.1 Schedule

A single task fires once daily at a configured hour. It always publishes the daily digest, and on one configured weekday it additionally publishes the weekly roll-up. Both are suppressed entirely when no reports channel is configured.

The scheduler has no catch-up for missed firings. An agent that is not running at the scheduled hour skips that day rather than publishing late on restart. A digest covering the previous day, delivered at an arbitrary hour two days later, is a distraction rather than a report.

9.2 Periodic status

Alongside the daily task, a second loop publishes a status snapshot at a configured interval, two hours by default, to the status channel. It reports the current state of every in-scope container and endpoint and is suppressed when no status channel is configured.

The snapshot is a level rather than an edge. It reports what is true at the moment it fires whether or not anything has changed, which is what distinguishes an idle channel from a broken one: a snapshot arriving on schedule showing every target healthy is positive evidence that the observer is alive, whereas silence from a transition-driven alerter is ambiguous between nothing happening and nothing working. The first snapshot fires on startup, which also confirms the scope the monitor came up with.

It renders the same current-state section the daily digest opens with, so a status post and the head of a digest cannot disagree.

9.3 Contents

The daily digest gives current state for every in-scope container and endpoint, followed by every transition recorded in the preceding twenty-four hours in chronological order.

The weekly roll-up gives the same current-state section, followed by per-target totals over the lookback window: how many separate outages the target had and how much cumulative downtime they represent.

9.4 Downtime accounting

Totals are computed by pairing failure events with the recovery events that follow them. Each unmatched failure opens an outage, and the matching recovery closes it and contributes its duration.

A failure with no following recovery means the target is still down. That outage is measured to the present moment and flagged accordingly, rather than being discarded for lack of a closing event. Discarding it would report the most serious case, a service that has been down for three days, as no downtime at all.

9.5 Measured data and generated prose

Digests separate two kinds of content, and the boundary is enforced rather than conventional.

The detail section is rendered directly from recorded events and current observations. The model does not see it before it is written and does not rewrite it. These are exact values, and passing

an exact value through a language model can only preserve it or damage it.

A summary section is generated on top of the rendered detail, giving a reader a short narrative before the numbers. If the model is unavailable, the summary degrades to the raw detail rather than failing the report.

The one exception

The weekly roll-up contains a narrative section that is genuinely model-generated. When a target has failed at least twice within the window, it is flapping, and the agent is asked to investigate that specific target: read its recent output, look at its resource behaviour, and give a short account of the likely cause.
This is the one question a template cannot answer. A template can count outages; it cannot read a log. If that investigation fails, the failure is caught and noted, and the rest of the report is published intact.

9.6 Publication and follow-up

A report is published as a unit. Where the target channel supports threads, the whole report becomes one thread rather than a series of loose messages. Long reports are split to respect the platform's per-message size limit.

The publication step returns the identifier of whatever container now holds the report, and the report body is written into that thread's conversation memory. An operator replying to the published report and asking why a service flapped is asking a question against a conversation that already contains the report.

10 The chat interface

10.1 Commands

Ten commands are registered through the chat library's application command tree [5, 6], listed in Table 3. Commands whose output concerns only the caller reply privately, so that routine list queries do not fill a shared channel.

Command	Argument	Effect
`status`	none	Current containers and endpoints, summarised
`investigate`	question	Free-form investigation with all operations
`report`	service	Structured incident report for one service
`digest`	period, publish flag	Render a scheduled report immediately
`containers`	none	Containers grouped by state
`watching`	none	Endpoints the monitor probes
`watch`	container	Add a container to the watch list
`unwatch`	container	Remove a container from the watch list
`watchlist`	none	Containers currently watched
`help`	none	List the commands

Table 3: The command set.

10.2 Conversational access

A direct message, or a mention in a channel the agent can see, is treated as a question and routed through the same investigation path as the explicit command. The mention markup is stripped before the text reaches the model. A mention carrying no question receives a prompt for one rather than an investigation of an empty string.

Because the conversation identifier is the channel, an operator can hold an ordinary exchange: ask a broad question, then ask why, then ask for the evidence, without restating the subject.

10.3 Output shaping

Model output requires two corrections before it is fit to send.

1. **Markdown dialect.** The platform renders a restricted dialect. Heading syntax is displayed literally rather than rendered, and tables are not supported at all, appearing as rows of pipe characters. Small models emit both regularly even when instructed not to. Headings are rewritten to bold text and tables are converted into labelled bullet lines. Content inside code fences is exempt, so that a log line containing a pipe character or a configuration comment beginning with a hash is not corrupted.
2. **Message length.** The platform caps message length. Answers exceeding it are split, preferring line boundaries so that a split does not fall inside a word or a code block.

10.4 Log forwarding

Application logs at informational level and above are forwarded to a configured channel, which makes the agent observable through the same interface it serves. Two details make this safe.

1. **Library records are dropped.** Records emitted by the chat library itself are not forwarded, since forwarding them would produce a feedback loop: sending a message logs an event, which sends a message.

2. **Forwarding is asynchronous.** Log emission places the record on a queue and returns immediately; a separate task drains the queue and sends. A logging call therefore never blocks on a network round trip, and a slow or unreachable channel degrades log delivery rather than stalling the code that logged.
3. **Records are batched.** The drain task collects for one second and packs what it finds into as few messages as the platform's length limit allows, rather than sending one message per record. A burst of activity costs one or two messages instead of a dozen, which keeps the forwarder clear of the per-channel rate limit that a message-per-line design walks straight into.
4. **The queue is bounded.** It holds a thousand records; beyond that, records are dropped. A log storm the channel cannot absorb costs visibility into that storm, which is recoverable, rather than unbounded memory in the process being observed, which is not.

What reaches the channel is the operational trail rather than debug noise: every monitored transition, every alert and report published, and for each question the origin channel, the operations the run called, and its wall-clock duration. Failures inside the drain task are written to standard error instead of logged, since logging a forwarding failure would feed the forwarder that failed.

On startup the bridge announces itself in the channel. An empty channel then means nothing is happening, rather than being indistinguishable from a broken configuration.

10.5 Failure must be visible

Message handling is deliberately structured so that no incoming question can be answered with silence.

The chat library's default handling of an exception raised inside an event handler is to log it against the library's own logger and continue. That logger is exactly the one excluded from channel forwarding to prevent the loop described above. The two behaviours compose badly: a failure while answering a mention would be logged where no operator was watching, and the visible result would be an agent that simply did not reply.

Handling is therefore placed in a workflow function wrapped in explicit exception handling that always sends a reply, stating that something went wrong and what. An error message is a worse answer than a correct one and a far better outcome than silence, because silence is indistinguishable from the agent being offline.

11 Configuration

All configuration is read from the process environment, optionally populated from an environment file, and validated once at startup into a typed object by a settings management library [7]. Table 4 groups the settings; Appendix A gives the full reference with defaults.

Group	Governs
Chat workspace	Authentication token, target workspace, and the four channel roles
Endpoint monitoring	Which endpoints are probed and how often
Scheduled reports	Publication hour, weekly day, lookback window
Language model	Provider, endpoint address, model names, credential
Observability backends	Metric and log server addresses and optional credentials
Container discovery	Runtime address, monitoring toggle, name filter
Loop limits	Operation budget, context budget, result and window caps
Storage	Location of the operational file; the conversation file sits beside it

Table 4: Configuration groups.

11.1 Required and optional

One setting is required: the chat workspace authentication token. Its absence is a startup failure with an explicit message rather than an obscure error later.

Several capabilities are gated on optional settings and stay off when those are unset. No alert channel means transitions are recorded but not announced. No reports channel means digests are not published. No log channel means logs go only to standard output. Container monitoring is off unless explicitly enabled.

Absence is treated as a configuration choice rather than an error. An operator who wants monitoring without notifications, or reports without a log bridge, expresses that by leaving the corresponding setting empty.

11.2 Handling of empty values

Optional numeric settings such as channel identifiers must distinguish unset from zero, so an empty environment variable is parsed as absent rather than as an empty string. Plain text settings need the opposite treatment, since an empty string is a valid value meaning no filter or no credential. Validation therefore restores empty strings for text settings after the general parsing rule has run.

The storage location is resolved once at import: the configured path is made absolute relative to the installation root, and its containing directory is created. The conversation file is derived from it rather than configured separately, so the two always sit together and no deployment can point them at different volumes.

11.3 Loop limits

Four settings bound resource use per question. The operation budget bounds how many operations one question may spend, as described earlier. The context budget sets the threshold at which history is compressed. A result limit bounds returned log lines. A window maximum bounds how far back a range query may reach.

The last two protect the backends rather than the model. A range query over a week of

high-volume log data is expensive to serve, and an investigation almost never needs it.

12 Deployment

12.1 Packaging

The agent is packaged as a single-service container image on a slim Python [1] base. Dependencies are resolved from a lock file [16] and installed in a layer built before the application source is copied, so that editing source does not invalidate the dependency layer. Builds after a source change reuse the cached dependency installation.

12.2 The observability backends

The deployment description [11] brings up the agent and nothing else. The metrics server and the log store it queries are treated as pre-existing infrastructure, addressed by configuration rather than started alongside it.

This follows from what the system is. It is a read-only client of an observability stack, not a distribution of one. A lab that already runs a metrics server has one configured, scraped and retained to suit itself, and a deployment description that brought up a second would either conflict with it or be ignored. Naming the addresses in configuration also means the two backends are independently optional: with either address unset the corresponding operations are absent from the tool set, and the agent answers from whatever remains.

The log store's labels are set by whatever ships logs into it, not by this system. Which labels exist is therefore a property of the deployment, and is exactly why the agent is given a discovery operation for label values rather than a fixed list. This is the concrete form of the point made in Section 6 about label schemas not being shared between the metrics and log backends.

12.3 Networking

The agent shares the host's network namespace, which lets it reach the observability backends and the external model endpoint at local addresses without further configuration. The consequence is that the bridge-networked components cannot address the agent by service name and reach it through the host gateway instead.

12.4 Persistence

The storage file is held on a mounted directory so that the watch list, the transition history and all conversation threads survive replacement of the container. Losing this directory loses the reporting history, which cannot be reconstructed from any other source.

12.5 Runtime socket access

The agent requires access to the container runtime's API socket to read container state and output. This deserves an accurate description, because a natural reading of the deployment description is misleading.

The socket is mounted with a read-only flag. That flag applies to the socket file. It does not constrain the API reachable through it: the runtime API remains fully writable, and access to that socket is equivalent to root on the host.

Where the guarantee actually comes from

The read-only guarantee rests entirely on the tool surface, described in Section 6.7. No operation available to the model performs a mutating call. The mount flag contributes nothing to that guarantee and should not be cited as though it did.
The operational consequence is ordinary: grant this access on the same basis as any other root-equivalent privilege on the host, and treat the containing host as the security boundary.

12.6 Operating the system

The process restarts automatically unless explicitly stopped. Shutdown closes the conversation store before the event loop ends, as described earlier, so that termination is prompt rather than requiring a forced kill.

For diagnosis, the log channel carries informational records and above in real time. The container's own output carries the same records and is the place to look when the chat connection itself is the problem. Transport-level logging is pinned to warnings, because informational logging of every backend request would emit a line per call and flood both destinations.

13 Verification

13.1 Automated tests

The test suite [17] runs entirely offline. It contacts no network service, requires no container runtime, no chat workspace and no model endpoint. This is a property of the architecture rather than of extensive mocking infrastructure. Because the agent builds its client through a single factory call, one fixture redirects that factory to a scripted substitute wherever it is used, and because the tool set is a named mapping, a test replaces one entry without disturbing the rest.

A fixture applied to every test redirects both the operational store and the conversation store to a temporary location and substitutes an in-memory conversation store. No test can reach production data even by mistake.

Coverage spans the areas where behaviour is not obvious from reading the code:

1. **Loop behaviour.** A turn that requests an operation before answering, a turn that answers immediately, an operation failure being fed back rather than raised, the operation budget halting the loop at exactly its limit, a model failure returning a message, a model that never answers hitting the outer time ceiling, and threads remaining isolated from one another.
2. **Conversation memory.** Thread configuration, and a thread surviving a closed and reopened connection, verified by asking a second question and checking that the first reached the model.
3. **Transition detection.** Both edges, the absence of an alert on first observation, scope changes pruning stale state, and an end-to-end incident scenario.
4. **Status publication.** A snapshot carrying the current-state section to the status channel, and its suppression when no status channel is configured.
5. **Log forwarding.** Records packing into as few messages as the length limit allows, an over-long record being truncated rather than dropped, and no message exceeding the limit.
6. **Report generation.** Outage pairing, unclosed-outage accounting, the flapping narrative and its absence when nothing flapped, weekday gating, and publication to the reports channel.
7. **Output shaping.** Heading and table conversion, exemption of fenced content, length capping, and the fallbacks when the model is unavailable or returns nothing.
8. **Name resolution.** Separator variants, exact-match precedence, and ambiguity raising.
9. **Backend clients.** Credential headers being sent only when configured, label validation rejecting malformed names, the log store's range delegation, and output frame parsing.
10. **The evaluation harness.** That the budget counts operations rather than turns, that a run which fails part way still writes its transcript, that an operation the model invents is reported as absent rather than broken, and that repeated runs are graded by the same rule as single ones.

Checks requiring live services are marked and excluded from the default run, so that the ordinary case needs no environment at all.

13.2 Model behaviour evaluation

Tests verify the code. They cannot verify that a given model chooses sensible operations, and a model substitution changes that materially while leaving every test passing. A separate notebook harness [18] addresses this, and is read by a human rather than asserted on.

Seven notebooks each isolate one subject operation, binding it, plus the discovery calls it requires,

and nothing else. All seven drive a shared instrumented loop rather than each rebuilding one, so a change to how runs are measured applies to every notebook at once. That loop is not the production harness: it is a deliberate reimplementation that records every message sent, every response, every operation and its result, and the token cost of each call, to a transcript file per run. Its budget counts operations exactly as the production harness does, so the two agree on how much work a question is allowed.

Each isolation notebook asks two questions. The first is answerable with the bound operations. The second requires evidence from a backend that is not bound, and is the more informative of the two: it measures whether the model states the limit or answers anyway. A model that reports a service healthy when it can only enumerate names has fabricated a verdict, and no automated test detects that.

Token consumption is accumulated per turn and reported, so each run shows what it cost.

An eighth notebook is different in kind: it imports the production agent itself and drives it with memory. It exercises what isolation cannot: which backend the model selects when nothing is withheld, and whether conversation context carries. Its sequence asks a broad question, then a follow-up naming nothing, which is answerable only if history carried forward, then repeats that follow-up on a fresh thread, which should fail, demonstrating isolation.

14 Design decisions

14.1 Read only by construction

- **Decision.** Expose no operation capable of changing anything.
- **Reasoning.** An agent that can act must be correct about when to act. An agent that can only observe must merely be useful. The gap between those two requirements is the difference between a system that can be deployed against a live environment now and one that needs an approval workflow, an audit trail and a rollback story first. Placing the constraint in the tool surface rather than in the instructions makes it verifiable by inspection.
- **Cost.** Every remediation is manual. An operator reading a correct diagnosis at three in the morning still restarts the service by hand.

14.2 The framework's harness, not a hand-written loop

- **Decision.** Build the agent with the framework's documented factory and configure its middleware, rather than writing the tool-calling loop, the budget, the error handling and the history trimming by hand.
- **Reasoning.** All four behaviours were implemented here first, and all four were ordinary. Maintaining them meant owning bugs in code that is not this project's subject, and the hand-written versions had already drifted: the budget was expressed in graph steps rather than operations, and trimming bounded the prompt while leaving the stored history unbounded. The framework's versions are maintained, documented and correct on both counts.
- **Cost.** The system now tracks a dependency's release cycle in a place where it previously tracked nothing, and behaviour that used to be readable in this repository is now a configuration flag whose meaning lives in someone else's documentation.

14.3 Failure is always answered

- **Decision.** Every failure path terminates in a message to the operator.
- **Reasoning.** Operation failures become model input. Model failures become a fallback message. Budget exhaustion becomes a partial summary. Alert dispatch failures are logged rather than raised, because they occur inside recurring tasks that a raised exception would silently stop, converting one failed alert into permanent monitoring loss. Message handling failures produce an explicit reply, for the reason given in Section 10.5.
- **Cost.** Failures are less conspicuous in aggregate. A backend that has been unreachable for a day produces a series of individually reasonable answers noting that it could not be checked, rather than one loud failure.

14.4 Measured data, generated prose

- **Decision.** Report figures are rendered from recorded data and never passed through the model. Only summary narrative is generated.
- **Reasoning.** The figures are already exact. A language model can preserve an exact number or corrupt it, and cannot improve it. Restricting generation to the parts where fluency helps, and excluding it from the parts where precision matters, takes the benefit without the risk.
- **Cost.** Reports contain a section of dense rendered detail that reads less smoothly than a fully generated report would.

14.5 Two storage files

- **Decision.** Operational state and conversation state live in separate files, each with exactly one opener.
- **Reasoning.** They shared one file while a published report had to become conversation context, which required the publishing and answering components to see the same threads. That path was removed, and what remained was two connections contending for one write lock for no benefit. One writer per file means a watch-list write cannot block a model turn.
- **Cost.** A backup is now two files rather than one, and anything wanting a consistent view of both has to take them together.

14.6 One name-matching rule

- **Decision.** Every lookup uses the same separator-insensitive rule, with exact match taking precedence and ambiguity raising.
- **Reasoning.** Consistency is the point. An operator who learns that partial names work when querying should find that they work identically when managing the watch list. Raising on ambiguity trades a round trip for the impossibility of a confidently mislabelled answer.
- **Cost.** An operator with several similarly named services types more.

15 Limitations and extensions

15.1 Limitations

1. **Answer quality tracks model quality.** The system supplies evidence and structure; the reasoning is the model's. A smaller model will discover names correctly and still draw a weak conclusion from them. The evaluation harness exists precisely because this cannot be tested automatically.
2. **Memory does not span conversations.** A thread is a conversation, and nothing is recalled across threads. An incident diagnosed in one channel contributes nothing to a similar incident raised in another.
3. **Long conversations keep a summary, not a record.** Once a thread exceeds the context budget its early turns are replaced by generated prose. A detail mentioned once and never repeated survives only if the summarising model judged it worth keeping.
4. **Monitoring has a blind interval.** Live state is in memory and the first observation after a restart is a baseline, so a failure occurring while the agent is down is never announced. It remains visible in the next report's current-state section.
5. **Missed report firings are skipped.** There is no catch-up, so an agent down at the scheduled hour omits that day's digest entirely.
6. **The deployment is single tenant.** There is no per-user authorisation beyond what the chat workspace enforces. Anyone who can message the agent can query every service it can see.
7. **Alerting observes state, not thresholds.** A service that is running but degraded, serving slowly or returning errors, produces no alert unless an HTTP probe happens to detect it.

15.2 Possible extensions

Several extensions follow naturally from the existing structure.

1. **Threshold alerting on metric data** would fill the degraded-but-running gap. It would need care to avoid the correlated-failure problem described in Section 8.5, most likely by suppressing metric-derived alerts when the metrics server itself is the failing target.
2. **Persisting monitor state** would close the blind interval, letting the agent compare its first observation after a restart against the last observation before it and report a failure that occurred while it was down.
3. **Write operations**, if ever added, should be introduced as a distinct tool set behind an explicit approval step rather than by adding mutating calls to the existing surface. The value of the guarantee in Section 6.7 lies in it being absolute. An agent that can act sometimes requires reasoning about every case in which it might.
4. **Additional backends** fit the existing pattern directly. A new backend is a client, a set of query operations with descriptions written for the model, and an entry in the tool list. Nothing in the agent changes.

16 Conclusion

The system automates a specific, repetitive part of operations work: gathering evidence from three separate systems and correlating it into an account of what happened. It does this through a language model given a fixed set of query operations, reached from the chat workspace an operator already uses.

Its most consequential property is what it cannot do. No operation available to the model changes anything, which places the guarantee in the composition of the tool set rather than in instructions the model may disregard. That single constraint is what allows an autonomous reasoning loop to run against a live environment while the worst available outcome remains an incorrect answer.

The remaining design follows from taking failure seriously in a system whose central component is probabilistic:

- **The reasoning loop is bounded**, and returns a partial summary instead of an error when the bound is reached.
- **Operation failures become evidence** rather than exceptions.
- **Report figures are rendered from recorded data** and never passed through the model, which restricts generation to the parts where fluency helps.
- **Monitoring baselines before it alerts**, because an alert stream that cries wolf on every restart is worse than none.

Every one of these choices costs something, and Section 14 states the costs alongside the reasoning.

References

[1] Python Software Foundation, "Python 3.13 documentation." [Online]. Available: https://docs.python.org/3.13/. [Accessed: Aug. 18, 2026].

[2] LangChain, Inc., "LangChain Python documentation: short-term memory." [Online]. Available: https://docs.langchain.com/oss/python/langchain/short-term-memory. [Accessed: Aug. 25, 2026].

[3] LangChain, Inc., "LangChain Python documentation: agents." [Online]. Available: https://docs.langchain.com/oss/python/langchain/agents. [Accessed: Aug. 25, 2026].

[4] OpenAI, "Chat Completions API reference." [Online]. Available: https://platform.openai.com/docs/api-reference/chat. [Accessed: Aug. 18, 2026].

[5] Rapptz and contributors, "discord.py documentation." [Online]. Available: https://discordpy.readthedocs.io/. [Accessed: Aug. 18, 2026].

[6] Discord, Inc., "Discord Developer Portal: API reference." [Online]. Available: https://discord.com/developers/docs/. [Accessed: Aug. 18, 2026].

[7] Pydantic Services, Inc., "Pydantic Settings documentation." [Online]. Available: https://docs.pydantic.dev/latest/concepts/pydantic_settings/. [Accessed: Aug. 18, 2026].

[8] Encode OSS Ltd., "HTTPX: a next-generation HTTP client for Python." [Online]. Available: https://www.python-httpx.org/. [Accessed: Aug. 18, 2026].

[9] SQLite Consortium, "SQLite documentation." [Online]. Available: https://www.sqlite.org/docs.html. [Accessed: Aug. 18, 2026].

[10] Docker, Inc., "Docker Engine API reference." [Online]. Available: https://docs.docker.com/reference/api/engine/. [Accessed: Aug. 18, 2026].

[11] Docker, Inc., "Docker Compose documentation." [Online]. Available: https://docs.docker.com/compose/. [Accessed: Aug. 18, 2026].

[12] Prometheus Authors, "Prometheus HTTP API." [Online]. Available: https://prometheus.io/docs/prometheus/latest/querying/api/. [Accessed: Aug. 18, 2026].

[13] Prometheus Authors, "Querying Prometheus: PromQL basics." [Online]. Available: https://prometheus.io/docs/prometheus/latest/querying/basics/. [Accessed: Aug. 18, 2026].

[14] Grafana Labs, "Grafana Loki HTTP API." [Online]. Available: https://grafana.com/docs/loki/latest/reference/loki-http-api/. [Accessed: Aug. 18, 2026].

[15] Grafana Labs, "LogQL: Loki query language." [Online]. Available: https://grafana.com/docs/loki/latest/query/. [Accessed: Aug. 18, 2026].

[16] Astral Software, Inc., "uv: Python packaging documentation." [Online]. Available: https://docs.astral.sh/uv/. [Accessed: Aug. 18, 2026].

[17] pytest Development Team, "pytest documentation." [Online]. Available: https://docs.pytest.org/. [Accessed: Aug. 18, 2026].

[18] Project Jupyter, "Jupyter documentation." [Online]. Available: https://docs.jupyter.org/. [Accessed: Aug. 18, 2026].

A Settings reference

Setting	Default	Meaning
Chat workspace		
Bot token	none	Authentication credential. Required; startup fails without it
Guild identifier	unset	Target workspace. When set, commands register there directly
Alert channel	unset	Receives failure notifications
Status channel	unset	Receives recovery notifications and the periodic snapshot
Reports channel	unset	Receives scheduled digests. Unset disables them
Log channel	unset	Receives forwarded application logs
Endpoint monitoring		
Monitored endpoints	empty	Comma-separated name and URL pairs
Monitor interval	60 s	Poll cycle period, floored at 10 seconds
Scheduled reports		
Report hour	05 UTC	Hour at which the daily task fires. Expressed in UTC, not local time
Weekly weekday	0 (Monday)	Day on which the weekly roll-up also publishes
Lookback days	7	Window covered by the weekly roll-up
Status interval	2 h	Period of the status snapshot, floored at one hour
Language model		
Provider	google	Selects the OpenAI-compatible or the Gemini family
Model name	configured	Model identifier requested from that endpoint
Summariser model	empty	Cheaper model for context compression. Empty reuses the main model
Base URL	local default	Address of the OpenAI-compatible endpoint
API key	empty	Credential, where the endpoint requires one
Gemini key	empty	Credential for the Gemini provider
Observability backends		
Metrics URL	local default	Address of the metrics server
Log store URL	local default	Address of the log store
Metrics token	empty	Bearer credential, sent only when set
Log store token	empty	Bearer credential, sent only when set
Container discovery		
Runtime address	local socket	Runtime API location
Monitoring enabled	false	Whether containers are polled at all
Name filter	empty	Scope substring. Empty places every container in scope

continued on next page

Setting	Default	Meaning
Loop limits		
Max tool steps	10	Operations permitted per question, counted per call
Model request timeout	30 s	Ceiling on one model request
Investigation timeout	60 s	Ceiling on a whole question, retries included
Max context tokens	12000	Threshold above which history is compressed
Log result limit	500	Maximum log lines returned by one query
Range maximum	6 h	Furthest back a range query may reach
Storage		
Database URL	local file	Location of the operational file. The conversation file is derived to sit beside it

B Operation reference

Operation	Arguments	Behaviour and limits
`list_services`	none	Container names plus configured endpoint names. Output capped. Returns a placeholder when the runtime is unreachable
`docker_status`	container name	State, status and image. Name matched loosely. Reports ambiguity rather than choosing
`docker_logs`	container name, line count	Recent combined output. Line count capped at 200. Stream framing parsed and non-printable bytes removed. Output capped
`list_metric_names`	optional filter	Available metric names, optionally filtered by substring. Entry count capped
`get_label_values`	label name	Values of one metric label. Label name validated before use. Entry count capped
`query_prometheus`	query expression	Instant query for current values. Output capped
`query_prometheus_range`	query expression, minutes	Range query. Window clamped to the configured maximum. Resolution derived from window length for a roughly constant point count. Output capped
`get_loki_label_values`	label name	Values of one stream label. Label name validated before use. Entry count capped. Labels are those the log shipper defines
`query_loki`	query expression, limit	Log query over a short recent window. Implemented as a range query because the instant endpoint rejects plain stream selectors. Limit clamped. Output capped
`query_loki_range`	query expression, minutes, limit	Log query over an explicit window. Window and limit both clamped. Output capped

Common properties

Every operation returns text and bounds what it returns. Every range and limit argument is clamped rather than rejected, so an over-large request yields usable data instead of an error. No operation accepts a parameter that would change state, and no operation is routed to a mutating endpoint.